



Informe técnico · Innovatech Solutions

Asignatura: DSY1106 · Desarrollo Full Stack

Autor: Najeeb Escobar Pérez

Fecha: Junio 2026

Contenidos

1. Contexto	3
2. Arquitectura	3
2.1 Contexto (C1)	3
2.2 Contenedores (C2)	3
2.3 Componentes (C3)	4
3. Stack tecnológico	4
4. Cómo se conectan las capas	5
5. Patrones de diseño	5
5.1 Repository	6
5.2 Factory Method	6
5.3 Circuit Breaker	6
5.4 API Gateway	6
5.5 Backend for Frontend (BFF)	7
5.6 Service Discovery (Eureka)	7
6. Modelo de datos	7
7. Decisiones críticas	8
8. Validación en Kubernetes	9

1. Contexto

Innovatech Solutions es una plataforma interna para gestionar proyectos, el talento asignado a esos proyectos y los indicadores que la dirección necesita para decidir. Resuelve un problema concreto: la información de proyectos, personas y capacidad vivía dispersa en planillas, sin una vista única ni control de acceso por rol. La plataforma centraliza esos datos y los muestra según quién mira.

Hay tres perfiles de usuario, y cada uno ve algo distinto:

- **Desarrollador (DEV):** sus tareas asignadas y los proyectos en los que participa.
- **Project Manager (PM):** los proyectos que supervisa, las tareas en riesgo y los hitos próximos.
- **Directivo (DIR):** la foto agregada de la organización (proyectos activos, porcentaje de utilización del equipo y alertas globales).

El dominio se reparte en tres capacidades: ciclo de vida de proyectos y tareas, gestión de recursos (las personas y su disponibilidad semanal) y analítica (KPIs de capacidad y avance). Esa separación por capacidad es la que define después la separación en microservicios.

El detalle de actores y sistemas externos está en el [Diagrama C1 · Contexto del Sistema](#).

2. Arquitectura

El sistema usa una arquitectura de microservicios sobre Spring Boot, con un frontend React de página única (SPA) que consume una API HTTP. La autenticación es externa, delegada en Keycloak. Todo corre en contenedores y se despliega en Kubernetes.

La descripción que sigue usa los tres primeros niveles del modelo C4: contexto (C1), contenedores (C2) y componentes (C3).

2.1 Contexto (C1)

En el nivel más alto hay un único sistema (Innovatech Solutions) con el que interactúan los tres perfiles de usuario a través del navegador. El sistema depende de un proveedor de identidad externo, Keycloak, que emite y firma los tokens de sesión. Ver [Diagrama C1 · Contexto del Sistema](#).

2.2 Contenedores (C2)

Dentro del sistema hay nueve contenedores ejecutables más cuatro bases de datos. Cada uno tiene una responsabilidad acotada:

Contenedor	Tecnología	Puerto	Responsabilidad
Frontend	React + Nginx	8080	SPA servida al navegador
API Gateway	Spring Cloud Gateway	9000	Entrada única, valida el token y enruta
BFF	Spring Boot MVC	8084	Orquesta los microservicios y arma la respuesta por rol
ms-projects	Spring Boot + JPA	8081	Proyectos y tareas

Contenedor	Tecnología	Puerto	Responsabilidad
ms-resources	Spring Boot + JPA	8082	Recursos (personas)
ms-analytics	Spring Boot + JPA	8083	Cálculo y captura de KPIs
Eureka Server	Spring Cloud Netflix	8761	Registro y descubrimiento de servicios
Keycloak	Keycloak 24	8080	Identidad y emisión de tokens
PostgreSQL x4	PostgreSQL 16	5432	Una base por servicio de negocio y una para Keycloak

El navegador nunca habla directo con un microservicio. Pasa por el gateway, y el gateway entrega al BFF, que es el único que conoce a los microservicios de negocio. El esquema completo de comunicación entre contenedores está en el [Diagrama C2 · Contenedores](#).

2.3 Componentes (C3)

El contenedor más interesante a nivel de componentes es el BFF, porque ahí está la orquestación. Sus componentes internos son:

- **Controladores** (`DashboardController` , `ProjectsController` , etc.): exponen la API que consume el frontend. Son delgados: solo mapean HTTP y delegan; la lógica de negocio vive en los servicios y la lectura del token (rol y email) en el DTO `AuthenticatedUser` .
- **Servicios de dominio** (`ProjectsService` , `ResourcesService` , `KpisService` , `TasksService`): cada uno envuelve la llamada a un microservicio dentro de un Circuit Breaker y define qué pasa si ese microservicio no responde.
- **Clientes HTTP** (`ProjectsClient` , `ResourcesClient` , etc.): resuelven la dirección del microservicio vía Eureka y ejecutan la llamada con `RestClient` .
- `DashboardDtoFactory`: arma el objeto de respuesta según el rol del usuario.

La descomposición interna del BFF está en el [Diagrama C3 · Componentes del BFF](#).

3. Stack tecnológico

Capa	Tecnología	Versión
Frontend	React	18.3.1
	TypeScript	5.6.3
	Vite	5.4.10
	Tailwind CSS	3.4.14
	Axios	1.7.7
	React Router	6.26.2
	Recharts	2.13.0
Backend	Java (JDK)	25

Capa	Tecnología	Versión
	Spring Boot	4.0.0
	Spring Cloud	2025.1.0
	Spring Cloud Gateway	incluido en Spring Cloud
	Eureka (Spring Cloud Netflix)	incluido en Spring Cloud
	Resilience4j	2.2.0
	SpringDoc OpenAPI	3.0.3
Persistencia	PostgreSQL	16
	Flyway (migraciones)	gestionado por Spring Boot
Identidad	Keycloak	24.0
Infraestructura	Nginx	alpine
	Ingress NGINX	controller v1.11.3
	Kubernetes	Docker Desktop (kind)

La elección apunta a un stack actual y coherente: todo el backend comparte versión de Spring Boot y Spring Cloud desde un POM padre, lo que evita conflictos de dependencias entre módulos.

4. Cómo se conectan las capas

El recorrido de una petición muestra cómo encajan las piezas. Tomemos el caso en que un directivo abre su dashboard.

1. El navegador carga la SPA desde `http://innovatech.localhost`, servida por el Ingress.
2. Si no hay sesión, el frontend pide un token a Keycloak con usuario y contraseña (flujo `*password grant*`). Keycloak valida contra su base y devuelve un JWT firmado, que el frontend guarda en `localStorage`.
3. El frontend llama a `GET http://localhost:9000/api/dashboard` y adjunta el token en la cabecera `Authorization: Bearer`.
4. El **API Gateway** recibe la llamada, valida la firma del token contra la clave pública de Keycloak (endpoint JWK), quita el prefijo `/api` y enruta hacia el BFF. La dirección del BFF la resuelve por Eureka, no está fija en configuración.
5. El **BFF** vuelve a validar el token (defensa en profundidad), extrae el rol y el email, y llama a los microservicios que necesita: a `ms-analytics` por los KPIs y a `ms-projects` por los proyectos. Cada llamada va envuelta en un Circuit Breaker.
6. Los **microservicios** consultan su propia base PostgreSQL con JPA y devuelven JSON.
7. El BFF arma el objeto del dashboard según el rol (un directivo no recibe lo mismo que un desarrollador) y responde.
8. El gateway devuelve la respuesta al navegador y React la pinta.

La autorización se valida en dos puntos: el gateway corta lo que llega sin token válido, y el BFF lo confirma antes de orquestar. Las rutas públicas (login, health, documentación) están explícitamente exceptuadas.

5. Patrones de diseño

5.1 Repository

Cada microservicio abstrae el acceso a su base con interfaces que extienden `JpaRepository`. La lógica de negocio trabaja contra esas interfaces y no contra SQL, lo que facilita las pruebas (se pueden simular) y deja que Spring Data genere las consultas a partir del nombre del método.

Ejemplo real en ms-resources (`ResourceRepository`):

```
public interface ResourceRepository extends JpaRepository<Resource, Long> {
    boolean existsByEmail(String email);
    Optional<Resource> findByEmail(String email);
}
```

Ver [Diagrama · Patrón Repository](#).

5.2 Factory Method

El BFF arma un dashboard distinto por rol. En vez de llenar el controlador de condicionales, `DashboardDtoFactory` decide qué objeto construir según el rol que viene en el token:

```
public DashboardDto create(UserRole role, String email) {
    return switch (role) {
        case PM -> pmDashboard(kpi, projects);
        case DEV -> devDashboard(projects, email);
        case DIR -> dirDashboard(kpi);
    };
}
```

El tipo de retorno es una interfaz sellada (`sealed interface DashboardDto`) con un **record** por cada variante. El compilador obliga a cubrir todos los roles, así que agregar un perfil nuevo es agregar un caso y el compilador avisa si falta. Ver [Diagrama · Patrón Factory Method](#).

5.3 Circuit Breaker

Las llamadas entre servicios pueden fallar, y sin protección un servicio caído arrastra a los demás. Con Resilience4j, cada llamada del BFF (y de ms-analytics) pasa por un Circuit Breaker configurado con ventana de 10 peticiones, umbral de fallo del 50%, tiempo límite de 3 segundos y 30 segundos en estado abierto antes de reintentar.

La parte importante es el **fallback**. Si ms-projects no responde, el BFF no revienta: devuelve una respuesta marcada como "datos no disponibles" con la lista vacía.

```
public ProjectsResponse list() {
    return breakers.create("projects").run(
        () -> ProjectsResponse.ok(client.list()),
        ex -> ProjectsResponse.unavailable());
}
```

Es una decisión deliberada: ante una caída, preferimos mostrar "no disponible" antes que arriesgar datos viejos sobre carga de personas o avance de proyectos. Ver [Diagrama · Patrón Circuit Breaker](#).

5.4 API Gateway

El gateway es el único punto de entrada público. Centraliza tres cosas que de otro modo habría que repetir en cada servicio: la validación del token, el CORS y el enrutamiento. Una sola ruta cubre toda la API:

```

routes:
  - id: bff-route
    uri: lb://bff          # resuelto por Eureka
    predicates:
      - Path=/api/**
    filters:
      - StripPrefix=1      # /api/dashboard -> /dashboard

```

Los servicios internos no conocen la URL pública ni el prefijo `/api` ; eso es un detalle que vive solo en el gateway. Ver [Diagrama · Patrón API Gateway](#).

5.5 Backend for Frontend (BFF)

El frontend necesita un dashboard que combina datos de tres microservicios. Sin BFF, el navegador tendría que hacer varias llamadas y armar el resultado en JavaScript. El BFF mueve esa orquestación al servidor: el frontend hace una sola llamada a `/dashboard` y recibe justo lo que tiene que pintar, ya filtrado por rol.

Además concentra la tolerancia a fallos (los Circuit Breakers viven acá) y la adaptación de formato. El frontend queda simple y el dominio de cada microservicio no se contamina con necesidades de la interfaz. Ver [Diagrama · Patrón BFF](#).

5.6 Service Discovery (Eureka)

Ningún servicio tiene direcciones de otros escritas a mano. Todos se registran en Eureka al arrancar, y cuando uno necesita a otro pregunta por nombre (`lb://bff` , `http://ms-projects`). Esto sostiene el despliegue en Kubernetes, donde las IP de los pods cambian, y deja la puerta abierta a correr más de una instancia de un servicio.

6. Modelo de datos

El sistema aplica **database per service**: cada microservicio de negocio tiene su propia base PostgreSQL y nadie entra a la base de otro. Keycloak tiene la suya aparte para usuarios y roles.

Servicio	Base	Entidad	Campos principales
ms-proj ects	projec ts	Projec t	id, name, description, status, startDate, plannedEndDate, ownerId
ms-proj ects	projec ts	Task	id, projectId, title, status, assigneeResourceId, estimatedHours, dueDate
ms-reso urces	resour ces	Resour ce	id, name, email (único), role, weeklyHours, skills, active
ms-anal ytics	analyt ics	KpiSna pshot	id, capturedAt, activeProjects, delayedProjects, totalActiveResources, weeklyCapacityHours, utilizationPercentage

Los estados son enumeraciones: un proyecto está en `PLANNING` , `IN_PROGRESS` , `COMPLETED` o `CANCELLED` ; una tarea en `TODO` , `IN_PROGRESS` , `DONE` o `BLOCKED` .

Las relaciones merecen una aclaración, porque hay dos tipos:

- **Relación física**, dentro de una misma base: una `Task` referencia a su `Project` con clave foránea real (`project_id`). Ambas viven en `projects` .
- **Relación lógica**, entre bases distintas: `Task.assigneeResourceId` apunta a un `Resource` que vive en otra base, así que no hay clave foránea; la referencia es solo el id. Lo mismo con `Project.ownerId` , que guarda el identificador del usuario de Keycloak.

ms-analytics no lee las bases de los otros. Para calcular los KPIs llama por HTTP a ms-projects y ms-resources, y guarda en su propia base los **snapshots** periódicos que alimentan los gráficos de tendencia.

El diagrama entidad-relación está en el [Diagrama · Modelo de Datos](#).

7. Decisiones críticas

Una base por servicio en vez de una base compartida. Cada microservicio es dueño de su esquema y puede evolucionar sin pedir permiso al resto. El costo es que no existen claves foráneas entre servicios y la consistencia entre ellos es eventual; se resuelve con referencias por id y comunicación HTTP. Se aceptó ese costo a cambio de la autonomía, que es el punto de tener microservicios.

ms-analytics consume por HTTP y no toca otras bases. Podría haber sido más rápido leer directo las tablas de proyectos y recursos, pero eso ataría el esquema de analytics a los esquemas ajenos. Al consumir por la API pública de cada servicio, un cambio interno en ms-projects no rompe analytics mientras el contrato HTTP se mantenga.

Fallback explícito "datos no disponibles". El Circuit Breaker no solo evita la cascada de fallos; define qué se muestra cuando algo se cae. La decisión fue no inventar ni reutilizar datos viejos, porque la plataforma informa decisiones sobre carga de personas y avance de proyectos. Un "no disponible" honesto es preferible a un número desactualizado.

Identidad delegada en Keycloak con validación en dos capas. No se programó manejo de contraseñas ni de sesiones; eso lo hace Keycloak, que emite JWT firmados. El gateway valida la firma y el BFF la vuelve a validar. Si el gateway llegara a fallar o alguien intentara saltárselo, el BFF sigue protegido.

CORS atado al origen real del despliegue. El frontend se sirve desde `innovatech.localhost` (Ingress), no desde `localhost:3000` . Tanto el gateway como el cliente de Keycloak deben permitir ese origen; si no, el navegador bloquea las llamadas y el registro o el login fallan de forma silenciosa. Fue un punto que costó detectar porque el error llegaba al usuario como un mensaje genérico.

Flyway para versionar el esquema. Las tablas no se crean por reflexión de Hibernate, sino con migraciones numeradas, y Hibernate queda en modo `validate` . Así el esquema es reproducible y el arranque falla rápido si el código y la base no calzan.

Bases como StatefulSet, no como Deployment. Las PostgreSQL son cargas con estado: necesitan identidad de red estable, arranque ordenado y un volumen propio que sobreviva al reinicio del pod. El StatefulSet da exactamente eso con `volumeClaimTemplates` (un volumen por réplica), mientras que los servicios sin estado quedan como Deployment. Es el tipo de objeto correcto si en algún momento se escala una base o se lleva a producción.

8. Validación en Kubernetes

El stack completo se despliega en un clúster local con Kustomize (`kubectl apply -k`). Cada servicio de aplicación corre como un Deployment con su Service; las cuatro bases PostgreSQL corren como StatefulSet, cada una con su volumen persistente (un PVC por réplica vía `volumeClaimTemplates`); la configuración no sensible va en un ConfigMap y las credenciales en un Secret. El frontend se publica con un Ingress en `innovatech.localhost` , y el gateway y Keycloak quedan accesibles por Service tipo LoadBalancer. La salud de cada pod se verifica con `*readiness probes*` contra `/actuator/health` .